

CHAPITRE 1

Écrire des scripts d'accès aux données

1. **Connexion à une base de données MySQL avec PDO**
2. Interrogation d'une base de données à travers un formulaire
3. Récupération des résultats
4. Manipulation des données CRUD



Introduction

- Pour enregistrer les données en PHP vous pouvez utiliser :
 - Les variables
 - Les fichiers
 - Une base de données
- Une base de données (BDD) permet de stocker, organiser et analyser les données.
- Un système de gestion de base de données (SGBD) permet d'accéder aux bases de données.

Introduction

- Une couche d'abstraction de base de données est une interface de programmation d'applications qui unifie la communication entre une application informatique et des bases de données.
- **L'API** : est une solution informatique qui permet à des applications de communiquer avec d'autres applications et de s'échanger mutuellement des services ou des données sans connaître les détails de leur mise en œuvre.
- **Une DLL** : Une Dynamic Link Library est une bibliothèque logicielle dont les fonctions sont chargées en mémoire par un programme, au besoin, lors de son exécution, par opposition aux bibliothèques logicielles statiques ou partagées dont les fonctions sont chargées en mémoire avant le début de l'exécution du programme. (source Wikipédia)

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



Les bases de données

CUBRID : Est une solution SGBDR (Système de Gestion de Base de Données Relationnelles) gratuite et open source.

dBase : Cette extension n'est pas recommandée par PHP et n'est plus intégrée depuis PHP 5.3.0

Firebird/InterBase : Cette extension n'est pas recommandée par PHP et n'est plus intégrée depuis PHP 7.4.0

IBM DB2 - Fonctions IBM DB2, Cloudscape et Apache Derby

MongoDB - MongoDB driver : Ces fonctions vous permettent un accès aux IBM DB2 Universal Database, IBM Cloudscape et Apache Derby qui utilisent DB2 Call Level Interface (CLI).

MySQL - Plugins et drivers MySQL : recommandé par PHP qui offre plusieurs drivers et plugins pour accéder et gérer MySQL.

OCI8 - Oracle OCI8 : Ces fonctions vous permettent d'accéder aux bases de données Oracle. Elles supportent les commandes SQL et PL/SQL.

PostgreSQL : Recommandé par PHP. La base de données PostgreSQL est un produit Open Source, disponible sans frais.

SQLite3 : Support des bases de données SQLite version 3.

SQLSRV - Driver Microsoft SQL Server pour PHP : Vous permet d'accéder à un serveur de base de données Microsoft SQL et SQL Azure.

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



MySQL

Avant de créer une connexion à un serveur de base de données MySQL, vous devez avoir :

- Un serveur de base de données MySQL installé sur votre système local ou sur un serveur distant.
- Une base de données sur le serveur MySQL.
- Un compte MySQL avec un nom d'utilisateur et un mot de passe permettant d'accéder à la base de données.
- Les plateformes de développement Web PHP comme EasyPHP, LAMP, WAMP, MAMP, LEMP et XAMPP

fournissent **MySQL. Fonctions MySQL en PHP:**

- `mysql_connect` — Ouvre une connexion à un serveur MySQL
- `mysql_create_db` — Crée une base de données MySQL
- `mysql_ping` — Vérifie la connexion au serveur MySQL, et s'y reconnecte au besoin
- `mysql_query` — Envoie une requête à un serveur MySQL
- ...
- Liste complète des fonctions MySQL en PHP : <https://www.php.net/manual/fr/ref.mysql.php>

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



Se connecter à MySQL en PHP

Pour pouvoir manipuler nos bases de données MySQL en PHP (sans passer par phpMyAdmin par exemple). PHP met à notre disposition deux API (Application Programming Interface) :

- L'extension MySQLi
- L'extension PDO (PHP Data Objects)
- La performance globale des deux extensions MySQLi et PDO peut être considérée comme identique. Nous allons choisir dans ce qui suit du cours l'interface PDO.

Interface d'abstraction pour le langage PHP :

- DBA : Couche d'abstraction de base de données (style dbm). Ces fonctions permettent l'accès aux bases de données du style Berkeley DB.
- ODBC - ODBC (Unifié) : vous donne accès à diverses bases de données qui ont emprunté la sémantique des API ODBC pour implémenter leur propres API. Les bases de données suivantes sont supportées par l'ODBC unifié : Adabas D, IBM DB2, iODBC, Solid, et Sybase SQL Anywhere.
- PDO : PHP Data Objects définit une excellente interface pour accéder à une base de données depuis PHP.

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



Installation PDO

- Choisissez les autres fichiers DLL spécifiques à votre base de données et utilisez soit la fonction `dl()` pour les charger au moment de l'exécution ou activez-les dans le fichier `php.ini` en-dessous de la ligne `php_pdo.dll`.

Fonction `dl` :

- **Définition** : Charge une extension PHP à la volée
 - **Syntaxe** : `dl(string $extension_filename) : bool`
-
- La librairie DLL doit exister dans le dossier système `extension_dir`.
 - Directive `extension_dir` : Spécifie le répertoire dans lequel PHP doit chercher des extensions externes à charger. Il est recommandé de spécifier un chemin absolu.
 - Il faut redémarrer PHP après chaque modification de votre `php.ini`, pour que les nouvelles directives de configuration prennent effet.
 - Lors de la compilation utilisez `--with-pdo-mysql[=DIR]` pour installer l'extension PDO MySQL, où `[=DIR]` est le chemin de la bibliothèque de base de MySQL

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



Fonctions PDO

PDO::beginTransaction → Démarre une transaction

PDO::commit → Valide une transaction

PDO::__construct → Crée une instance PDO qui représente une connexion à la base

PDO::errorCode → Retourne le SQLSTATE associé avec la dernière opération sur la base de données

PDO::errorInfo → Retourne les informations associées à l'erreur lors de la dernière opération sur la base de données

PDO::exec → Exécute une requête SQL et retourne le nombre de lignes affectées

PDO::getAttribute → Récupère un attribut d'une connexion à une base de données

PDO::getAvailableDrivers → Retourne la liste des pilotes PDO disponibles

PDO::inTransaction → Vérifie si nous sommes dans une transaction

PDO::lastInsertId → Retourne l'identifiant de la dernière ligne insérée ou la valeur d'une séquence

PDO::prepare → Prépare une requête à l'exécution et retourne un objet

PDO::query → Prépare et Exécute une requête SQL sans marque substitutive

PDO::quote → Protège une chaîne pour l'utiliser dans une requête SQL PDO

PDO::rollBack → Annule une transaction

PDO::setAttribute → Configure un attribut PDO

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



La classe PDOStatement

- PDOStatement : Représente une requête préparée et, une fois exécutée, le jeu de résultats associé.

Méthodes :

- **PDOStatement::bindColumn** : Lie une colonne à une variable PHP
- **PDOStatement::bindParam** : Lie un paramètre à un nom de variable spécifique
- **PDOStatement::bindValue** : Associe une valeur à un paramètre
- **PDOStatement::columnCount** : Retourne le nombre de colonnes dans le jeu de résultats
- **PDOStatement::debugDumpParams** : Détaille une commande préparée SQL
- **PDOStatement::errorCode** : Récupère les informations sur l'erreur associée lors de la dernière opération sur la requête
- **PDOStatement::errorInfo** : Récupère les informations sur l'erreur associée lors de la dernière opération sur la requête
- **PDOStatement::execute** : Exécute une requête préparée
- **PDOStatement::fetchAll** : Retourne un tableau contenant toutes les lignes du jeu d'enregistrements
- **PDOStatement::getAttribute** : Récupère un attribut de requête
- ...
- Liste complète : <https://www.php.net/manual/fr/class.pdostatement.php>

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



La classe PDOException

- **PDOException** : Représente une erreur émise par PDO.

Propriétés :

- **errorInfo** : Retourne les informations associées à l'erreur lors de la dernière opération sur la base de données. Correspond à PDO::errorInfo() ou PDOStatement::errorInfo().
- **errorCode** : Retourne le SQLSTATE associé avec la dernière opération sur la base de données. (SQLSTATE : un identifiant alphanumérique de cinq caractères défini dans le standard ANSI SQL.)
- **PDO::errorInfo**
 - **Syntaxe** : public PDO::errorInfo(): array
 - **Retourne un tableau contenant :**
 - [0] : Code d'erreur SQLSTATE
 - [1] : Code d'erreur spécifique au driver.
 - [2] : Message d'erreur spécifique au driver.
- **PDO::errorCode**
 - **Syntaxe** : public PDO::errorCode(): ?string
 - Retourne null si aucune opération n'a été exécutée sur la base de données.

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



Configuration

- Le pilote PDO_MYSQL implémente l'interface de PDO pour autoriser l'accès de PHP aux bases de données MySQL. Pour déclarer le pilote du SGBD MySQL, il faut ajouter la ligne suivante dans le fichier php.ini : extension=php_pdo_mysql.dll
- L'extension PDO peut utiliser soit la bibliothèque mysqlnd, soit la bibliothèque libmysqlclient.
- La bibliothèque interne mysqlnd est la bibliothèque par défaut et qui est recommandé par le PHP.

Nom	Défaut	Modifiable	Historique
pdo.dsn.*		php.ini seulement	

Tab.1 : Options de configuration PDO. Source : <https://www.php.net/manual/fr/pdo.configuration.php>

Nom	Défaut	Modifiable
pdo_mysql.default_socket	"/tmp/mysql.sock"	PHP_INI_SYSTEM
pdo_mysql.debug	NULL	PHP_INI_SYSTEM

Tab.2 : Options de configuration du driver PDO_MYSQL. Source : <https://www.php.net/manual/fr/ref.pdo-mysql.php>

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



SQL

- SQL (Structured Query Language) est un langage de programmation standardisé utilisé pour gérer les bases de données au sein des SGBD et plus particulièrement les SGBD relationnelles (ex: MySQL), permettant d'effectuer des opérations sur les données qu'elles contiennent.

Les instructions SQL couvrent 4 domaines :

- Langage de définition de données (LDD) : permet de créer ou supprimer des objets dans la base de données.
 - Langage de manipulation de données (LMD) : permet de manipuler les données contenues dans les tables.
 - Langage de contrôle de données (LCD) : permet de gérer les utilisateurs d'une base de données ainsi que leurs droits sur les objets.
 - Langage de contrôle des transactions (LCT) : permet de valider ou annuler des modifications de données dans la base de données.
- Liste des commandes SQL : <https://sql.sh/>
 - Une erreur à la ligne de la fonction fetchAll() est affichée lorsqu'une erreur de requête SQL est détectée dans un code PHP.

Exemple Erreur fetchAll() :

Fatal error: Call to a member function fetchAll() on a non-object in index.php on line 13

01 - Écrire des scripts d'accès aux données

Connexion à une base de données MySQL avec PDO



Exemple: Script de connexion à MySQL avec PDO

- Le serveur de base de données MySQL local.
- Une base de données nommée « test ».
- Le compte root avec un mot de passe vide.

```
<?php

$host = 'localhost';
$dbname = 'test';
$username = 'root';
$password = '';

try {

    $connexion = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    $connexion -> setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Vous êtes maintenant connecté à $dbname sur $host.";

} catch (PDOException $e) {
    die("Impossible de se connecter à la base de donnée $dbname :". $e->getMessage());
}
```

CHAPITRE 1

Écrire des scripts d'accès aux données

1. Connexion à une base de données MySQL avec PDO
2. **Interrogation d'une base de données à travers un formulaire**
3. Récupération des résultats
4. Manipulation des données CRUD



01 - Écrire des scripts d'accès aux données

Interrogation d'une base de données à travers un formulaire



Paramétrer la connexion

- Pour établir la connexion, il faut créer un DSN (Data Source Name) en renseignant des informations comme par exemple :
 - Le type de base de données qui est MySQL.
 - Le nom de la base de données.
 - Le port de connexion.
 - L'encodage.
 - ...

```
$dsn =  
'mysql:host=ADRESSE_DU_SERVEUR;dbname=  
VOTRE_BASE_DE_DONNEES;port=VOTRE_  
PORT;charset=VOTRE_ENCODING';
```

Connexion au serveur MySQL

- Pour connecter PHP à MySQL avec PDO, il est obligatoire de fournir les quatre renseignements suivants :
 - Le nom d'hôte
 - La base de données
 - L'identifiant (login)
 - Le mot de passe

Syntaxe :

```
$connexion_pdo = new PDO($dsn,  
'VOTRE_NOM_UTILISATEUR_SERVEUR' ,  
'VOTRE_MOT_DE_PASSE');
```

Exemple :

```
<?php  
$mysqlConnection = new PDO(  
'mysql:host=localhost;dbname=testDB;charset=utf8', 'root', 'root'  
);  
?>
```

01 - Écrire des scripts d'accès aux données

Interrogation d'une base de données à travers un formulaire



Connexion avec gestion des erreurs

- Pour afficher des détails sur l'erreur, il faut activer les erreurs lors de la connexion à la base de données via PDO.
- Activer les erreurs lors de la connexion à la base de données permettra d'avoir un message d'erreur détaillé.

Exemple :

```
<?php
$db = new PDO(
    'mysql:host=localhost;dbname=test;charset=utf8',
    'root',
    '',
    [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION],
);
?>
```

01 - Écrire des scripts d'accès aux données

Interrogation d'une base de données à travers un formulaire



Utiliser MySQL

- Création d'une base de données : `CREATE DATABASE test`

- Création d'un login et mot de passe : `CREATE USER 'user1'@'localhost' IDENTIFIED BY ".monPass1!"`

(On crée un utilisateur « user1 » avec le mot de passe « .monPass1! »)

- Création d'une table :

```
CREATE TABLE personne (  
  id int(11) unsigned NOT NULL auto_increment PRIMARY KEY,  
  nom varchar(30) NOT NULL,  
  email varchar(20) NOT NULL,  
  timestamp int(11) unsigned NOT NULL  
);
```

- Enregistrer les données en PHP : `$table = "personne";`

01 - Écrire des scripts d'accès aux données

Interrogation d'une base de données à travers un formulaire



Requêtes SQL

Ajouter une colonne : `ALTER TABLE [tablename] ADD COLUMN [colonne3] VARCHAR(100);`

Ajouter une colonne avec un ID unique par incrémentation automatique : `ALTER TABLE [tablename] ADD COLUMN [colonne4] int NOT NULL AUTO_INCREMENT PRIMARY KEY;`

Insérer un enregistrement dans une table SQL : `INSERT INTO [tablename] ([colonne1], [colonne2]) VALUES ('[valeur1]', '[valeur2]');`

01 - Écrire des scripts d'accès aux données

Interrogation d'une base de données à travers un formulaire



Formulaire HTML

- En HTML, il existe principalement deux méthodes pour interagir avec un utilisateur :
 - Les liens (balise <a>).
 - Les formulaires (balise <form>).

Exemple :

```
<html>
<head>
  <title>Exemple formulaire en PHP/MySQL</title>
</head>
<body>
  <form method="post" action="controller.php">
    Name : <input type="text" name="nom"
placeholder="Entrez votre nom" /><br />
    Email : <input type="email" name="email"
placeholder="Entrez votre Email" /><br />
    <input type="submit" value="Soumettre" />
  </form>
</body>
</html>;
```

Page traitement PHP

- La page PHP permet de collecter des données à partir du formulaire.
- La valeur de l'attribut « action » dans la balise de formulaire HTML signifie que toutes les valeurs des champs d'entrée seront envoyées au fichier .php présent dans la valeur de l'attribut « action ».

Exemple (**controller.php**) :

```
<?php
// Vérifie qu'il provient d'un formulaire
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  if (!isset($name)){
    die("S'il vous plaît entrez votre nom");
  }
  if (!isset($email)){
    die("S'il vous plaît entrez votre adresse e-mail");
  }
  print "Salut " . $name . "!, votre adresse e-mail est " . $email;
}
?>
```

01 - Écrire des scripts d'accès aux données

Interrogation d'une base de données à travers un formulaire



Stockage des données dans MySQL

Prérequis : une table « utilisateurs » créée dans la base de donnée MySQL.

```
<?php
//identifiants mysql
$host = "localhost";
$username = "root";
$password = "";
$dbdatabase = "utilisateurs";

$nom = $_POST["nom"];
$email = $_POST["email"];

//Ouvrir une nouvelle connexion au serveur MySQL
$connexionPDO = new PDO("mysql:host=$host;dbname=$dbdatabase", $username, $password);

//préparer la requête d'insertion SQL
$stmtement = $connexionPDO->prepare("INSERT INTO users (nom, email) VALUES(?, ?)");
//Associer les valeurs et exécuter la requête d'insertion
$stmtement->bindParam('ss', $nom, $email);
?>
```

CHAPITRE 1

Écrire des scripts d'accès aux données

1. Connexion à une base de données MySQL avec PDO
2. Interrogation d'une base de données à travers un formulaire
- 3. Récupération des résultats**
4. Manipulation des données CRUD



01 - Écrire des scripts d'accès aux données

Récupération des résultats



Requêtes SQL

Envoyez une requête SQL depuis votre script PHP en utilisant : `PDO::query()`

Afficher toutes les données d'une table : `SELECT * FROM [tablename];`

Sélectionner des enregistrements spécifiques : `SELECT * FROM [table] WHERE [colonne] = [valeur];`

Sélectionnez les enregistrements qui contiennent la valeur [val] : `SELECT * FROM [table] WHERE [colonne] LIKE '%[val]%' ;`

Sélectionnez les enregistrements commençant par 'val1' et se terminant par 'val2' : `SELECT * FROM [table] WHERE [colonne] LIKE '[val1_val2]';`

Sélectionnez des enregistrements sans doublons : `SELECT distinct nom, email FROM personne WHERE nom = "Mohammed";`

Faire un SELECT sur plusieurs tables : `SELECT [table1].[colonne1], [table2].[colonne2] FROM [table1], [table2];`

Constantes PDO

PDO::FETCH_ASSOC (int)

- Spécifie que la méthode de récupération doit retourner chaque ligne dans un tableau indexé par les noms des colonnes comme elles sont retournées dans le jeu de résultats correspondant. Si le jeu de résultats contient de multiples colonnes avec le même nom, PDO::FETCH_ASSOC retourne une seule valeur par nom de colonne.

PDO::FETCH_NAMED (int)

- Spécifie que la méthode de récupération doit retourner chaque ligne dans un tableau indexé par les noms des colonnes comme elles sont retournées dans le jeu de résultats correspondant. Si le jeu de résultats contient de multiples colonnes avec le même nom, PDO::FETCH_NAMED retourne un tableau de valeurs par nom de colonne.

PDO::FETCH_COLUMN (int)

- Spécifie que la méthode de récupération doit retourner uniquement une seule colonne demandée depuis la prochaine ligne du jeu de résultats.

- Liste des constantes prédéfinies : <https://www.php.net/manual/fr/pdo.constants.php>

CHAPITRE 1

Écrire des scripts d'accès aux données

1. Connexion à une base de données MYSQL avec PDO
2. Interrogation d'une base de données à travers un formulaire
3. Récupération des résultats
4. **Manipulation des données CRUD**



01 - Écrire des scripts d'accès aux données

Manipulation des données CRUD



CRUD

- CRUD est un acronyme des noms des quatre opérations de base de la gestion de la persistance des données et applications :
 - Create (créer)
 - Read ou Retrieve (lire)
 - Update (mettre à jour)
 - Delete ou Destroy (supprimer)
- Les opérations CRUD sont exécutées différemment en fonction de l'environnement, comme indiqué ci-dessous dans le tableau :

CRUD-Operation	SQL	RESTful HTTP	XQuery
Create	INSERT	POST, PUT	insert
Read	SELECT	GET, HEAD	copy/modify/return
Update	UPDATE	PUT, PATCH	replace, rename
Delete	DELETE	DELETE	delete

Tab.3 : Opérations CRUD dans SQL, RESTful HTTP et XQuery

01 - Écrire des scripts d'accès aux données

Manipulation des données CRUD



Requêtes SQL

Mettre à jour des enregistrements : `UPDATE [table] SET [colonne] = '[new_val]' WHERE [colonne] = '[old_val]';`



Supprimer des enregistrements : `DELETE FROM [table] WHERE [colonne] = [valeur];`



Supprimer tous les enregistrements d'une table : `DELETE FROM [table];`



Supprimer les colonnes d'une table : `ALTER TABLE [table] DROP COLUMN [colonne];`



Supprimer une table : `DROP TABLE [table];`



Supprimer une base de donnée : `DROP DATABASE [database];`



Créer un nouvel utilisateur : `CREATE USER 'username'@'localhost' IDENTIFIED BY 'password';`



Accorder un accès complet à l'utilisateur sur toutes les tables : `GRANT ALL PRIVILEGES ON database.* TO 'user'@'localhost';`

01 - Écrire des scripts d'accès aux données

Manipulation des données CRUD



Insérer plusieurs entrées dans une table

Exemple :

```
$dbco->beginTransaction();

$sql1 = "INSERT INTO Clients(Nom,Prenom,Pays,Mail)
        VALUES('Ahmed','Ben','Maroc','test@gmail.com)";
$dbco->exec($sql1);

$sql2 = "INSERT INTO Clients(Nom,Prenom,Pays,Mail)
        VALUES('Julia','Joly','France','test@yahoo.com)";
$dbco->exec($sql2);

$dbco->commit();
echo 'Entrées ajoutées dans la table';
```

01 - Écrire des scripts d'accès aux données

Manipulation des données CRUD



Exemple organisation des fichiers PHP

index.php :

Page d'accueil de notre application CRUD.

create.php :

Créez de nouveaux enregistrements avec un formulaire HTML et envoyez des données au serveur avec une requête POST.

read.php :

Affichez les enregistrements de notre table de base de données et naviguez avec la pagination.

update.php :

Met à jour les enregistrements existants avec un formulaire HTML et envoie les données au serveur avec une requête POST.

delete.php :

Confirme et supprime les enregistrements par ID (requête GET pour obtenir l'ID).

functions.php :

Fonctions de base de modèles et fonction de connexion MySQL (nous n'avons donc pas à répéter le code dans chaque fichier).

style.css :

La feuille de style de notre application, cela changera l'apparence de notre application.